

TechSpec_BookingModel_v2

BookingModel.com — Technical Specification

Version: 2.0 | **Date:** August 2026 | **Owner:** VEA Group — VEA Tech **Stack:** Next.js 15 · Supabase · Stripe · Cloudinary · Resend · OpenAI

1. OVERVIEW & POSITIONING

BookingModel.com là **KOL/KOC booking marketplace** — kết nối brand/agency với food & lifestyle creators đã được VEA vet thủ công. Khác với self-serve platforms: VEA curate từng creator, verify contact info, và đảm bảo deal execution.

Market focus (Phase 1): Food & Lifestyle creators — TikTok/YouTube/Instagram, US market, \$80-2,500/deal.

Three-layer architecture:

bookingmodel.com	→ Public marketplace (browse, apply, signup)
bookingmodel.com/login	→ Brand dashboard (contact info, brief, deal track)
admin.bookingmodel.com	→ VEA internal (creator DB, import, deals, applicants)

2. DATABASE SCHEMA

2.1 Users

```
users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email       TEXT UNIQUE NOT NULL,  
  password_hash TEXT,  
  full_name   TEXT,  
  role        TEXT DEFAULT 'brand'  
              CHECK (role IN ('creator', 'brand', 'agency', 'admin')),  
  company_name TEXT,  
  stripe_customer_id TEXT,  
  plan        TEXT DEFAULT 'free'  
              CHECK (plan IN ('free', 'standard', 'pro')),  
  is_verified BOOLEAN DEFAULT FALSE,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
)
```

2.2 Creators

```
creators (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID REFERENCES users(id),          -- NULL if admin-added  
  name        TEXT NOT NULL,  
  handle      TEXT UNIQUE NOT NULL,  
  platform    TEXT NOT NULL,                    -- 'TikTok', 'YouTube', 'Instagram'  
  channel_url TEXT,  
  niche       TEXT,  
  audience    TEXT,                             -- '82K', '1.2M'  
  er          TEXT,                             -- '6.2%'  
  rate_min    INTEGER,                          -- USD cents  
  rate_max    INTEGER,  
  contact_email TEXT,  
  contact_verified BOOLEAN DEFAULT FALSE,  
  contact_notes TEXT,  
  bio         TEXT,  
  photo_url   TEXT,                             -- Cloudinary URL  
  avatar_url  TEXT,                             -- fallback  
  status      TEXT DEFAULT 'pending'  
              CHECK (status IN ('pending', 'active', 'inactive', 'rejected')),  
  source      TEXT DEFAULT 'manual'
```

```

import_batch_id    CHECK (source IN ('manual','csv_import','self_apply','crawl')),
                    UUID,
bd_notes           TEXT,                                -- internal BD notes
created_at         TIMESTAMPTZ DEFAULT NOW(),
updated_at         TIMESTAMPTZ DEFAULT NOW()
)

```

2.3 Creator Portfolio

```

creator_portfolio (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id    UUID REFERENCES creators(id) ON DELETE CASCADE,
  type          TEXT CHECK (type IN ('image','video')),
  url           TEXT NOT NULL,                          -- Cloudinary or YouTube URL
  thumbnail     TEXT,                                    -- Cloudinary URL
  youtube_id    TEXT,                                    -- for video embeds
  label         TEXT,
  sort_order    INTEGER DEFAULT 0,
  created_at    TIMESTAMPTZ DEFAULT NOW()
)

```

2.4 Campaigns

```

campaigns (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  brand_id      UUID REFERENCES users(id),
  title         TEXT NOT NULL,
  category      TEXT,                                    -- 'Food','Beauty','Fitness'
  content_type  TEXT,                                    -- 'UGC Video','Testimonial'
  platform      TEXT,
  spots_total   INTEGER DEFAULT 5,
  spots_filled  INTEGER DEFAULT 0,
  budget_usd    INTEGER,
  brief_text    TEXT,
  status        TEXT DEFAULT 'draft'
                CHECK (status IN ('draft','active','paused','completed')),
  published_at  TIMESTAMPTZ,
  created_at    TIMESTAMPTZ DEFAULT NOW()
)

```

2.5 Deals (Bookings)

```

deals (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  deal_ref      TEXT UNIQUE,                            -- 'BM-2026-001'
  campaign_id   UUID REFERENCES campaigns(id),
  creator_id    UUID REFERENCES creators(id),
  brand_id      UUID REFERENCES users(id),
  content_type  TEXT,
  deliverables  TEXT,                                    -- '3x UGC Video 30sec'
  value_usd     INTEGER,                                 -- in cents
  platform_fee  INTEGER,                                 -- 15% of value_usd
  status        TEXT DEFAULT 'negotiating'
                CHECK (status IN (
                  'negotiating','brief_sent','content_in_review',
                  'approved','published','completed','overdue','cancelled'
                )),
  due_date      DATE,
  stripe_payment_intent_id TEXT,
  escrow_released BOOLEAN DEFAULT FALSE,
  notes         TEXT,
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  updated_at    TIMESTAMPTZ DEFAULT NOW()
)

```

2.6 Applicants (KOL/KOC Self-Registration)

```

applicants (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name          TEXT NOT NULL,
  handle        TEXT,
  platform      TEXT,
  channel_url   TEXT,
  niche         TEXT,

```

```

audience    TEXT,
er           TEXT,
rate        TEXT,
email       TEXT NOT NULL,
phone       TEXT,
photo_url   TEXT,
video_url   TEXT,                                -- portfolio video
notes       TEXT,
status      TEXT DEFAULT 'pending'
CHECK (status IN ('pending','approved','rejected')),
admin_notes TEXT,
creator_id  UUID REFERENCES creators(id),        -- set when approved
applied_at  TIMESTAMPTZ DEFAULT NOW()
)

```

2.7 Partnership Requests (Hợp Tác)

```

partnership_requests (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT,
  company     TEXT,
  email       TEXT NOT NULL,
  phone       TEXT,
  type        TEXT,                                -- 'VEA Media','VEA Tech' etc
  description TEXT,
  status      TEXT DEFAULT 'new'
CHECK (status IN ('new','in_discussion','converted','closed')),
assigned_to  UUID REFERENCES users(id),
source       TEXT DEFAULT 'website',              -- 'website','veagroup'
created_at   TIMESTAMPTZ DEFAULT NOW()
)

```

2.8 Import Batches

```

import_batches (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  filename    TEXT,
  total_rows  INTEGER,
  imported    INTEGER,
  duplicates  INTEGER,
  errors      INTEGER,
  status      TEXT,
  imported_by UUID REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT NOW()
)

```

3. API ROUTES (Next.js App Router)

3.1 Auth

```

POST /api/auth/signup    → create user + send verification email
POST /api/auth/login     → Supabase auth → return session
POST /api/auth/logout    → clear session
GET  /api/auth/me        → return current user

```

3.2 Creators (Public)

```

GET /api/creators        → list active creators (paginated, filtered)
                             ?niche=food&platform=TikTok&min_er=5&max_rate=500&page=1&limit=20
GET /api/creators/:id    → creator public profile + portfolio

```

3.3 Creators (Brand — requires auth)

```

GET /api/creators/:id/contact → full contact info (gated by plan)
POST /api/creators/:id/save   → save to favorites

```

3.4 Creators (Admin)

```

GET  /api/admin/creators    → full DB including inactive
POST /api/admin/creators    → create creator manually
PUT  /api/admin/creators/:id → update creator

```

DELETE /api/admin/creators/:id → soft delete (status='inactive')
POST /api/admin/creators/import → bulk import from CSV/Excel
body: { rows: CreatorRow[], batch_name: string }
response: { imported, duplicates, errors, batch_id }

3.5 Applicants

POST /api/applicants → public KOL/KOC registration form
GET /api/admin/applicants → list all applicants (admin)
PUT /api/admin/applicants/:id → approve/reject + optional create creator

3.6 Deals

GET /api/admin/deals → all deals
POST /api/admin/deals → create deal
PUT /api/admin/deals/:id → update status, notes

3.7 Partnership Requests

POST /api/partnership → public hợp tác form
GET /api/admin/partnership → admin list
PUT /api/admin/partnership/:id → update status, assign

3.8 Campaigns

GET /api/campaigns → public active campaigns
POST /api/admin/campaigns → create campaign (admin/brand)
PUT /api/admin/campaigns/:id → update

3.9 Media Upload

POST /api/upload/photo → upload to Cloudinary → return URL
multipart/form-data: file
POST /api/upload/video-thumb → fetch YouTube thumbnail → store Cloudinary
body: { youtube_url: string }

4. EXCEL/CSV IMPORT SYSTEM

4.1 Supported Formats

- .csv (UTF-8, comma-separated)
- .xlsx / .xls (via SheetJS)
- .tsv (tab-separated)

4.2 Required Columns

name	TEXT	Creator display name
handle	TEXT	@handle (unique key for dedup)
platform	TEXT	TikTok YouTube Instagram Facebook
channel_url	TEXT	Full URL
audience	TEXT	e.g. "82K" or "82000"
er	TEXT	e.g. "6.2%" or "6.2"
rate	TEXT	e.g. "\$150-350"
niche	TEXT	e.g. "Healthy Meal Prep / Budget"
contact	TEXT	Email or contact hint
notes	TEXT	BD notes
photo_url	TEXT	Direct image URL (Unsplash, Cloudinary, or any CDN)
video_url	TEXT	YouTube URL for portfolio video

4.3 Import Flow

1. Admin uploads file → /api/admin/creators/import
2. Server: parse CSV/Excel → validate columns
3. For each row:
 - a. Check handle uniqueness → skip if exists
 - b. Validate required fields (name, handle, platform)
 - c. If photo_url: store in Cloudinary (re-host)
 - d. If video_url (YouTube): extract video_id, fetch thumbnail → Cloudinary
 - e. Insert into creators + creator_portfolio
4. Return: { imported: N, duplicates: M, errors: [...] }

5. Admin sees preview table → confirm or cancel

4.4 Photo & Video Handling

```
// Photo: re-host to Cloudinary for performance
async function reHostPhoto(url: string): Promise<string> {
  const result = await cloudinary.uploader.upload(url, {
    folder: 'bookingmodel/creators',
    transformation: [{ width: 400, height: 440, crop: 'fill', gravity: 'face' }],
  });
  return result.secure_url;
}

// Video: extract YouTube ID, fetch thumbnail
function extractYouTubeId(url: string): string | null {
  const match = url.match(/(?:youtube\.com\/(?:watch?v=|shorts\/)|youtu\.be\/)([^\n?]+)/);
  return match ? match[1] : null;
}

async function fetchYouTubeThumbnail(videoId: string): Promise<string> {
  const thumbUrl = `https://img.youtube.com/vi/${videoId}/maxresdefault.jpg`;
  return await reHostPhoto(thumbUrl); // re-host to Cloudinary
}
```

5. CREATOR REGISTRATION FLOW

5.1 Public Form Fields

name *	TEXT	
handle *	TEXT	e.g. @myhandle
platform *	SELECT	TikTok YouTube Instagram Facebook Other
channel_url *	URL	
niche *	TEXT	
audience	TEXT	approximate follower count
er	TEXT	engagement rate if known
rate	TEXT	rate expectation
email *	EMAIL	
phone	TEXT	
photo_url	URL	optional – link to profile photo
video_url	URL	optional – YouTube portfolio link
notes	TEXT	anything else to share

5.2 Processing

1. Submit → POST /api/applicants
2. Store in applicants table (status='pending')
3. Send confirmation email to applicant (Resend)
4. Send notification to admin (Resend)
5. Admin reviews in Internal dashboard → Applicants tab
6. Admin: Approve → creates creator profile OR Reject → notify applicant
7. Approved creator → status='active' → visible on public marketplace

6. MEDIA SYSTEM

6.1 Photos

- Storage: **Cloudinary** (free tier: 25GB, 25K transformations/mo)
- Transformations on upload: w_400,h_440,c_fill,g_face for creator cards
- Fallback: Unsplash URL (current demo) → migrate to Cloudinary in production
- Admin can update photo from creator detail panel

6.2 Videos

- Embed: **YouTube oEmbed** — admin/creator provides YouTube URL
- Thumbnail: auto-fetch from YouTube → re-host to Cloudinary
- Player: YouTube iframe embed (no custom player needed in v1)
- Future: direct video upload → Cloudinary Video (up to 100MB/file)

6.3 File Size Limits

Creator portrait:	max 5MB, convert to WebP
Portfolio images:	max 10MB each
CSV/Excel import:	max 50MB (10K rows per batch)
Video thumbnail:	fetches from YouTube (no size limit)
Direct video:	max 100MB (Phase 2 feature)

7. AUTH & PERMISSIONS

7.1 Roles

Role	Access
admin	Full internal dashboard, all CRUD, import, deals, settings
brand (Pro)	Browse creators + contact info, brief builder, deal tracking, unlimited searches
brand (Standard)	Browse creators, 10 contact reveals/day, brief builder
brand (Free)	Browse creators only (no contact info)
creator	Own profile management, portfolio upload, deal status

7.2 Plan Gates

```
function canViewContact(user: User, creator: Creator): boolean {
  if (user.role === 'admin') return true;
  if (user.plan === 'pro') return true;
  if (user.plan === 'standard') {
    // Check daily contact reveal count
    return getDailyRevealCount(user.id) < 10;
  }
  return false; // free plan
}
```

8. PAYMENTS (Stripe)

8.1 Subscriptions

Free:	\$0/mo	– browse only
Standard:	\$49/mo	– 10 contacts/day, brief builder
Pro:	\$149/mo	– unlimited contacts, deal tracking, priority support

8.2 Deal Escrow

- Brand creates deal → pays into Stripe escrow
 - Creator delivers content → brand approves
 - Stripe releases payment to creator (minus 15% BookingModel fee)
 - Creator payout: Stripe Connect (direct to bank)
-

9. EMAIL NOTIFICATIONS (Resend)

Trigger	Recipients	Template
Creator applies	Admin + Creator	Application received
Application approved	Creator	Welcome + profile link
Application rejected	Creator	Feedback + reapply option
New deal created	Creator + Brand	Deal brief + timeline
Deal status changed	Both parties	Status update
Content submitted	Brand	Review request
Deal completed	Both	Completion + payment release
New partnership request	Admin	Lead notification
Brand signup	Brand	Welcome + onboarding

10. PERFORMANCE & SCALE

10.1 Targets (Phase 1)

- Page load: < 1.5s (LCP)
- Creator search: < 200ms
- CSV import (1,000 rows): < 30s
- Uptime: 99.9%

10.2 Caching

```
// Creator list: cache 5 minutes (Supabase + Next.js ISR)
export const revalidate = 300; // 5 minutes

// Individual creator: cache 1 hour
export const revalidate = 3600;

// Admin views: no cache (always fresh)
```

10.3 Infrastructure

Frontend: Vercel (auto-scale, edge network)
Database: Supabase (PostgreSQL, connection pooling via PgBouncer)
Media CDN: Cloudinary (global CDN, auto-format WebP/AVIF)
Email: Resend (100K emails/mo free tier)

11. SECURITY

- Auth: Supabase Auth (JWT, httpOnly cookies, PKCE flow)
 - Admin route: middleware check role === 'admin' → redirect 403
 - File upload: server-side type validation + Cloudinary virus scan
 - Rate limiting: Upstash Redis (contact reveals, form submissions)
 - CORS: restricted to bookingmodel.com and admin.bookingmodel.com
 - RLS: Supabase Row Level Security on all tables
-

12. DEPLOYMENT

12.1 Environment Variables

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Stripe
STRIPE_SECRET_KEY=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
STRIPE_WEBHOOK_SECRET=

# Cloudinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# Resend
RESEND_API_KEY=

# OpenAI (Phase 2)
OPENAI_API_KEY=

# Admin
ADMIN_EMAIL=admin@bookingmodel.com
```

12.2 Vercel Config

```
{
  "rewrites": [
    { "source": "/admin/:path*", "destination": "/admin/:path*" }
  ],
  "headers": [
    {
```

```
    "source": "/admin/:path*",
    "headers": [{ "key": "X-Robots-Tag", "value": "noindex" }]
  }
]
```

13. PHASED ROLLOUT

Phase	Timeline	Features
Demo (current)	✓ Done	HTML demo, 40 creators, Unsplash photos, localStorage
Phase 1	4-6 weeks	Deploy on Vercel, Supabase backend, real auth, CSV import with Cloudinary
Phase 2	8-12 weeks	Brand subscriptions (Stripe), deal escrow, email notifications
Phase 3	3-4 months	Self-registration live, AI matching, mobile app (React Native)

TechSpec v2.0 — BookingModel.com — VEA Group · VEA Tech Prepared by VEA Tech team · August 2026